

RESEARCH SYNTHESIS

Layers of a Persistent World

Rendering, operative state, provenance, and committed history

CENTRAL PRINCIPLE

History is authoritative at commitment boundaries; state is authoritative between them.

Flyxion

Independent Researcher

September 2026

Abstract

Research on persistent multiplayer worlds, learned world models, distributed simulation, formal transition systems, artificial life, and operational state-space geometry converges on a common problem: a world must remain identifiable while computation, observation, and participation continually change it. Yet these literatures often use the word state for several different things. This report separates four layers: rendered appearance, operative physical or simulated state, derived provenance and permission state, and append-only committed history. It argues that rendering and numerical evolution are disposable computations; that an observation becomes historical only when it is attested and constrains future admissibility; that current physical and provenance states should be certified folds over one authoritative event history; and that refusals and replay discrepancies must be appended rather than erased. The resulting architecture clarifies why visual continuity is not world continuity, why latent compression is not persistent state, and why current state alone cannot carry identity, ownership, or provenance. The report locates each layer in established research while marking the synthesis itself as a new construction rather than an existing quantum-game-world theory.

1. The layers question

A simulated world is easy to mistake for whatever appears on screen. Learned video systems intensify that mistake because they can generate visually convincing continuations without maintaining an explicit, queryable world state. Classical engines make the opposite simplification: they identify the world with the current values of objects, fields, inventories, and coordinates. Persistent and multiplayer systems expose a third requirement. A world must also remember which changes were admitted, which were refused, who acted, what evidence constrained the result, and how the present acquired its authority.

The research corpus supports a sharp initial distinction between a rendering latent and a state latent. GameNGen-like systems condition future images on prior frames and actions; PlaNet, Dreamer, MuZero, IRIS, and related systems learn iterable predictive structure that can support control or planning. But even an explicit state-transition model does not by itself preserve historical identity. A model can reproduce the same present state from incompatible pasts, silently replace one account with another, or restore a snapshot that looks correct while losing the commitments by which the state became legitimate.

rendered appearance $\neq$ operative state $\neq$ provenance state $\neq$ committed history

These are not four independent worlds. They are four roles within one architecture. Rendering presents a view. Operative state supports ongoing dynamics. Provenance state summarizes historically relevant relations such as ownership, attribution, access, and standing constraints. Committed history is the durable record from which authoritative states are derived.

2. A four-layer architecture

Layer	Primary object	Authority	Typical failure
Rendering	Observer-relative appearance x	None beyond presentation	Visual continuity mistaken for world continuity
Operative state	Fields, objects, variables M	Provisional dynamics	Snapshot treated as complete identity
Provenance state	Ownership, permissions, attributions Q	Certified historical projection	Context recomputed informally or lost
Committed history	Typed event DAG H	Identity and admissibility	Past silently patched after divergence

Table 1. The layers differ by function and authority; they need not use different storage systems.

2.1 Rendering

Rendering computes what a particular observer can presently receive. It may depend on viewpoint, visibility, level of detail, uncertainty, access rights, and presentation medium. It is intentionally disposable: repeated render calls should not enlarge history merely because more observers looked. Rendering can be cached, approximated, regenerated, or degraded without changing what the world has committed to.

$$x_t = R(M_t, Q_t, \omega_t)$$

The observer parameter ω_t includes location, sensory channel, credentials, and other view conditions. The renderer reads certified current folds rather than scanning the complete event history at every frame.

2.2 Operative state

The operative state M_t contains what the dynamics need now: positions, velocities, fields, inventories, health values, object relations, and other variables used by transition rules. Between commitments it may evolve through many provisional solver steps. These steps matter computationally but need not each become part of historical identity.

$$M\text{-tilde}(t + dt) = F_{dt}(M\text{-tilde}(t); \theta, \xi)$$

Here θ identifies rules or solver versions and ξ records controlled stochastic input. The tilde marks a provisional state that can be recomputed or refined before any semantic commitment is made.

2.3 Provenance and permission state

The provenance fold Q_t carries relations that affect interpretation and admissibility without necessarily changing immediate physics: ownership, authorship, repair lineage, warrants, access rules, sanctions, unresolved claims, and scars understood as attributions. Keeping Q separate prevents physical state from becoming an unstructured warehouse for every historical relation while avoiding an unbounded history scan during ordinary operation.

2.4 Committed history

The authoritative history H is an append-only collection of typed events. In a single-authority system it may be a sequence. In multiplayer or distributed settings it is more naturally a directed acyclic graph whose edges record causal dependence. A committed event states not merely that a value changed, but that a candidate passed or failed an admissibility boundary under identifiable evidence and rules.

3. One history, several folds

The cleanest reconciliation is to treat M and Q as certified caches produced by different folds over the same history. This makes history authoritative without requiring every frame or numerical microstep to replay the entire past.

$$M_t = \text{fold}(\text{commit_phys}, M_0, H_t)$$

$$Q_t = \text{fold}(\text{commit_prov}, Q_0, H_t)$$

A physical commit function applies events that alter operative dynamics. A provenance commit function applies events that alter standing, attribution, access, or interpretation. Other folds may be added for accounting, narrative summaries, scientific measurement, or observer-specific indexes. The world is therefore not simply a snapshot paired with a log. It is the authoritative log together with a family of certified folds and checkpoints.

RESOLUTION

History is authoritative at commitment boundaries. State is authoritative between them only as the current certified or provisional result of admissible evolution.

4. Observation is not attestation

Observation must not be used as a loose synonym for commitment. If every view were an event, history would grow with observer count and rendering would acquire unintended causal power. Merely looking is a disposable computation. An observation becomes historical only when it is registered as evidence that constrains the admissible set.

$$\text{looking: } R(M_t, Q_t, \omega) \rightarrow x \quad [\text{no commit}]$$

attesting: $(x, \text{witness}, \text{rule}) \rightarrow E_{(t+1)} \rightarrow \text{narrower admissible set}$

This distinction aligns simulation engineering with an epistemic account of evidence. Sensors may sample continually while only selected, authenticated, or threshold-crossing measurements become attestations. A screenshot need not alter a world; a signed measurement used to authorize a repair can. The decisive property is not perceptual occurrence but constraining work.

5. Admission, refusal, and repair

A history containing only successful transitions is incomplete. Failed proposals disclose boundaries, attacks, conflicts, and changing expectations. The event vocabulary should therefore include at least proposed, admitted, refused, repaired, and replay-discrepant outcomes. A refusal is not a physical transition, but it is still an event about the relation between an attempted action and the world's constraints.

candidate $c \rightarrow \text{gate } A(c \mid M_t, Q_t, H_t) \rightarrow \{\text{admit, refuse, repair}\}$

The physical fold normally ignores a refusal because no admitted physical change occurred. The provenance fold may record it because an attempted intrusion, invalid transfer, or prohibited repair can alter permissions, suspicion, audit status, or future admissibility. Repair should likewise remain visible: replacing a failed proposal with a corrected one must not erase the distinction between what was first attempted and what was finally admitted.

6. Checkpoints and replay divergence

Checkpoints are necessary for large worlds, but they must be treated as accelerators rather than rival authorities. A checkpoint binds a folded state to a history frontier, ruleset, solver identity, seed policy, and verification hash. It permits efficient recurrence while preserving the claim that the state is certified by a particular history.

$C_k = (\text{frontier}_k, M_k, Q_k, \text{versions}_k, \text{seeds}_k, \text{hash}_k)$

Exact bitwise replay may fail because of nondeterministic solvers, floating-point differences, hardware changes, missing dependencies, or corrupted artifacts. The framework's policy is simple: do not patch the earlier checkpoint or rewrite its event. Append a replay-discrepancy event containing the expected and obtained hashes, environment description, and resulting disposition. Later repair may create a new certified checkpoint while the failed recurrence remains part of provenance.

This policy separates epistemic immutability from operational rigidity. The system may improve its solver, restore service, or adopt a better reconstruction. What it may not do is present the improved result as though it had always been the original result.

7. Continuous fields without infinite history

A history-primary account need not record every timestep of a continuous field. The distinction is between numerical work and semantic commitment. Solver ticks approximate a trajectory; semantic events identify candidate changes; commits determine which changes have become identity-bearing facts.

solver ticks -> semantic candidates -> committed events

A field-level commit can record the initial or prior certified state, boundary conditions, forcing data, solver and operator versions, stochastic seeds, tolerance policy, interval, constraint checks, and resulting checkpoint hash. Intermediate values can be retained for diagnostics or discarded. What must survive is enough information to state what was asserted, under which procedure, and with what verification result.

Commit frequency is task-relative. A collision, transfer of ownership, public attestation, branching decision, or irreversible phase change may deserve a commit even when it occurs inside a much finer numerical integration. Conversely, thousands of stable substeps may remain provisional. The history records distinctions that matter to recurrence and accountability, not an indiscriminate transcript of computation.

8. Distributed worlds and causal history

Persistent multiplayer worlds cannot assume a single total order known everywhere at once. Interest management gives participants partial and sometimes stale projections; network delay creates concurrent proposals; rollback and reconciliation revise provisional states. Chandy and Lamport's distributed-snapshot result marks an important epistemic limit: a globally consistent snapshot need not correspond to any instantaneous state that every process jointly occupied.

The natural historical structure is therefore a DAG. Each commit names the causal frontier on which it depends. Concurrent commits may be folded in either order only when their effects commute and neither changes the admissibility of the other.

$$e_i \text{ independent of } e_j \text{ implies } \text{fold}(e_i \text{ then } e_j) = \text{fold}(e_j \text{ then } e_i)$$

This is the operational meaning of the residue-algebra principle that combination commutes only over causally independent commitments. When two actions compete for one object, depend on the same scarce resource, or alter each other's permissions, the system must order, refuse, branch, or repair them. It cannot declare them independent for convenience.

9. What the literature establishes

9.1 Persistent worlds

State-machine replication establishes that deterministic operations applied in agreed order can define a replicated service. Virtual Time supplies rollback, antimeessages, and fossil collection for distributed simulation. MMO persistence research distinguishes live state management from durable persistence and shows that different consistency classes impose different logging and replay obligations. Ceptre gives a particularly close formal analogue: world state is a multiset of resources, transitions consume and produce resources, permanent facts are marked separately, and execution traces admit causal-dependency structure.

9.2 Learned world models

World Models, PlaNet, Dreamer, and MuZero treat learned dynamics as iterable structure supporting prediction, planning, or policy learning. Neural video engines demonstrate action-conditioned visual continuation but often lack an explicit persistent state. The literature therefore supports the distinction between compression latents and state latents. Recent memory mechanisms add poses, timestamps, geometry, or retained visual context, confirming that persistence must be engineered rather than expected to emerge from rendering quality.

9.3 Formal state and admissibility

POMDPs supply state-dependent action sets, partial observability, belief states, and noninvertible transition kernels. Structural operational semantics defines admissible transitions as those derivable from rules. Computational mechanics defines causal states as equivalence classes of histories with the same predictive futures, placing history and continuation in direct formal relation. Viability theory studies the set of initial states from which constraints can continue to be satisfied; reachability theory studies which states can be attained under controlled dynamics. Event structures give irreversibility a particularly clear home because configurations grow while conflict permanently excludes alternatives.

9.4 Artificial life and social simulation

Schelling, Boids, Sugarscape, Tierra, Avida, Neural MMO, Generative Agents, and related systems demonstrate how local rules and per-agent state generate persistent global organization. Their scientific importance is methodological: simulation becomes an instrument for discovering consequences not transparent from the rules alone. This supports the use of computational worlds as epistemic laboratories without invoking the metaphysical claim that physical reality is literally an externally executed simulation.

9.5 Quantum and information geometry

Generalized probabilistic theories, quantum instruments, categorical quantum mechanics, monotone information metrics, and Markov categories provide rigorous languages for states, transformations, interventions, distinguishability, and composition. They do not establish a direct quantum account of MMOs or neural game engines. Their legitimate role is comparative and structural: quantum theory is a highly developed operational geometry whose methods illuminate how a theory can define admissible states and transformations. Markov categories form the strongest bridge to ordinary stochastic transition systems.

10. What is new in the synthesis

No located literature already combines persistent-world engineering, learned world models, provenance-sensitive rendering, append-only refusal-aware history, and operational geometry into one framework. The contribution is therefore not the discovery that games are quantum systems. It is the transfer of a shared formal vocabulary across substrates while respecting the differences between them.

The main new claim is that persistent worldhood requires more than controllable state. A system becomes historically persistent when current folds are certified by an append-only record of admitted, refused, repaired, and discrepant events; when observation is separated from attestation; when checkpoints accelerate but do not overwrite recurrence; and when concurrency is resolved according to causal dependence rather than retrospective convenience.

This produces a hierarchy of increasingly strong claims. Renderability means that an appearance can be generated. Controllability means that actions produce structured transitions. Recurrence means that a state can be reproduced under declared conditions. Provenance means that the legitimacy and lineage of the state can be recovered. Persistence means that identity survives change because commitments and refusals remain addressable.

11. Design rules for an implementation

Keep rendering pure. A render call may read certified folds but should not write history merely because a view was produced.

Commit attestations, not perceptions. Evidence enters history only when an identified procedure uses it to constrain admissibility.

Use typed events. Distinguish proposals, admissions, refusals, repairs, attestations, checkpoints, and replay discrepancies.

Derive current authority. Physical and provenance states are folds or certified checkpoints tied to explicit history frontiers.

Make refusal durable. A failed proposal may leave physical state unchanged while still affecting audit and permission folds.

Never repair the past in place. A corrected reconstruction or checkpoint is appended with its relation to the failed one.

Record causal parents. Distributed commits name their dependencies so independent events can commute and conflicting ones cannot.

Separate recurrence levels. Bitwise, numerical, semantic, and provenance equivalence must not be collapsed into one replay claim.

12. Research questions

1. What is the smallest event vocabulary sufficient to reconstruct both operative and provenance folds without making every solver step historical?
2. Which admissibility predicates must be evaluated against current folds, and which require direct reference to an historical witness?
3. When may two events commute in a distributed history, and how can independence be certified rather than assumed?
4. How should replay equivalence be graded across bitwise, numerical, semantic, causal, and provenance-preserving recurrence?
5. Can a learned world model acquire persistent identity without an explicit event ledger, or does apparent persistence merely relocate the ledger into opaque memory?
6. How should refusal affect future permissions without allowing adversarial attempts to inflate or poison the authoritative history?
7. Can viability kernels be defined over the combined folded state (M, Q), making institutional and provenance constraints part of reachable future volume?
8. Which information metric best measures the loss incurred when a historically distinct world is collapsed into an extensionally identical snapshot?

Conclusion

The layers question is resolved by assigning different kinds of authority to different representations. Rendering governs appearance but creates no history. Operative state governs provisional evolution between commitment boundaries. Provenance and permission state summarize historically active relations. Committed history governs identity, auditability, and the legitimacy of recurrence. One event history can support many efficient folds, so historical authority need not imply continuous log scanning or the storage of every numerical microstep.

The decisive boundary is not between computation and storage but between provisional work and attested constraint. Looking is not an event; attesting is. A refusal can be historically real without producing a physical change. A failed replay can be repaired without rewriting what failed. A snapshot can accelerate reconstruction without becoming the source of truth. In this architecture, a world persists not because it renders the same scene again, but because it can state what was committed, what was refused, and why the present is entitled to count as a continuation of its past.

Selected references

- Aubin, J.-P. (1991). *Viability Theory*. Birkhauser.
- Barrett, J. (2007). Information processing in generalized probabilistic theories. *Physical Review A*, 75, 032304. <https://doi.org/10.1103/PhysRevA.75.032304>
- Chandy, K. M., & Lamport, L. (1985). Distributed snapshots: Determining global states of distributed systems. *ACM Transactions on Computer Systems*, 3(1), 63-75. <https://doi.org/10.1145/214451.214456>
- Coecke, B., & Kissinger, A. (2017). *Picturing Quantum Processes*. Cambridge University Press. <https://doi.org/10.1017/9781316219317>
- Crutchfield, J. P., & Young, K. (1989). Inferring statistical complexity. *Physical Review Letters*, 63, 105-108. <https://doi.org/10.1103/PhysRevLett.63.105>
- Epstein, J. M. (1999). Agent-based computational models and generative social science. *Complexity*, 4(5), 41-60.
- Fritz, T. (2020). A synthetic approach to Markov kernels, conditional independence and theorems on sufficient statistics. *Advances in Mathematics*, 370, 107239. <https://doi.org/10.1016/j.aim.2020.107239>
- Ha, D., & Schmidhuber, J. (2018). World Models. *arXiv:1803.10122*.
- Hafner, D., Lillicrap, T., Fischer, I., Villegas, R., Ha, D., Lee, H., & Davidson, J. (2019). Learning latent dynamics for planning from pixels. *Proceedings of ICML*. *arXiv:1811.04551*.
- Hafner, D., Lillicrap, T., Ba, J., & Norouzi, M. (2020). Dream to Control: Learning behaviors by latent imagination. *Proceedings of ICLR*. *arXiv:1912.01603*.
- Jefferson, D. R. (1985). Virtual time. *ACM Transactions on Programming Languages and Systems*, 7(3), 404-425. <https://doi.org/10.1145/3916.3988>
- Kaelbling, L. P., Littman, M. L., & Cassandra, A. R. (1998). Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1-2), 99-134.
- Martens, C. (2015). Ceptre: A language for modeling generative interactive systems. *Proceedings of AIIDE*.
- Schneider, F. B. (1990). Implementing fault-tolerant services using the state machine approach. *ACM Computing Surveys*, 22(4), 299-319. <https://doi.org/10.1145/98163.98167>
- Schrittwieser, J., et al. (2020). Mastering Atari, Go, chess and shogi by planning with a learned model. *Nature*, 588, 604-609. <https://doi.org/10.1038/s41586-020-03051-4>
- Shalizi, C. R., & Crutchfield, J. P. (2001). Computational mechanics: Pattern and prediction, structure and simplicity. *Journal of Statistical Physics*, 104, 817-879.
- Winsberg, E. (2010). *Science in the Age of Computer Simulation*. University of Chicago Press.